

KoRo Data Science Internship

SQL Assessment Submission

| **Submitted by:** Swari Tamboli | swari.tamboli@gmail.com

| **Date:** March 18, 2026

| **Stack:** BigQuery SQL · Python (pandas, matplotlib, seaborn)

Dataset Overview

The assessment uses four tables hosted in BigQuery under the `koRo_Aufgabe` dataset:

Table	Description	Key Columns
<code>orders</code>	Individual order records	order_id, customer_id, order_date, product_number, country_iso
<code>marketing_sources</code>	Marketing channel attribution	order_id, country_iso, reporting_channel, utm_campaign
<code>product_locale</code>	Localized product names	sku, product_name, locale
<code>product_universal</code>	Universal product categories	sku, main_category

Data notes: All queries handle NULLs explicitly. `SAFE_DIVIDE` is used throughout to avoid division-by-zero errors. `ROW_NUMBER()` window functions are used for first-order identification. `DENSE_RANK()` is used for product ranking to correctly handle ties.

Task 1 Customer Order Activity & New Customer Share Over Time

Objective: Understand early customer behavior and measure what share of daily orders come from first-time buyers.

Approach

The query is structured as three CTEs:

1. `customer_order_ranked` : Assigns a row number to each order per customer, sorted by date ascending, so `order_rank = 1` identifies the very first order.
2. `first_orders` : Filters down to only the first order per customer, capturing the `first_order_date`.
3. `customer_activity` : Joins all orders back to their first-order date, computes `days_since_first_order`, and flags whether each order is the customer's first (`is_first_order = 1`).

The final `SELECT` aggregates by `order_date` to produce daily metrics.

SQL Query (BigQuery)

```
WITH customer_order_ranked AS (  
  SELECT  
    order_id,  
    customer_id,  
    order_date,  
    product_number,  
    country_iso,  
    ROW_NUMBER() OVER (  
      PARTITION BY customer_id  
      ORDER BY order_date ASC  
    ) AS order_rank  
  FROM `koRo_Aufgabe.orders`  
  WHERE order_date IS NOT NULL  
    AND customer_id IS NOT NULL  
) ,  
  
first_orders AS (  
  SELECT  
    customer_id,  
    order_date AS first_order_date  
  FROM customer_order_ranked  
  WHERE order_rank = 1  
) ,  
  
customer_activity AS (  
  SELECT
```

```

        o.order_id,
        o.customer_id,
        o.order_date,
        o.product_number,
        o.country_iso,
        f.first_order_date,
        DATE_DIFF(o.order_date, f.first_order_date, DAY) AS day
s_since_first_order,
        CASE WHEN DATE_DIFF(o.order_date, f.first_order_date, D
AY) = 0
                THEN 1 ELSE 0 END                                AS is_
first_order
    FROM `koRo_Aufgabe.orders` o
    JOIN first_orders f USING (customer_id)
    WHERE o.order_date IS NOT NULL
)

SELECT
    order_date,
    COUNT(order_id)    AS total_orders,
    SUM(is_first_order) AS new_customer_orders,
    ROUND(
        SAFE_DIVIDE(SUM(is_first_order), COUNT(order_id)) * 10
0, 2
    )    AS new_customer_pct,

    -- Early repeat activity within 10 / 15 / 20 days of firs
t order
    COUNTIF(days_since_first_order BETWEEN 1 AND 10) AS repea
t_orders_d10,
    COUNTIF(days_since_first_order BETWEEN 1 AND 15) AS repea
t_orders_d15,
    COUNTIF(days_since_first_order BETWEEN 1 AND 20) AS repe
at_orders_d20

FROM customer_activity
GROUP BY order_date
ORDER BY order_date;

```

Key Design Decisions

- `ROW_NUMBER()` (not `RANK`) is used so each customer has exactly one row with rank 1, even if two orders share the same timestamp.
 - `SAFE_DIVIDE` prevents division-by-zero on dates with zero total orders.
 - NULLs in `order_date` and `customer_id` are filtered early in the first CTE to keep downstream logic clean.
 - The repeat windows (D10, D15, D20) use `BETWEEN 1 AND N` to exclude day 0 (the first order itself).
-

Task 1 (Bonus) Channel Breakdown by Reporting Channel

Objective: Add a `reporting_channel` dimension from `marketing_sources` to understand which acquisition channels drive the most new customers.

Approach

The `marketing_sources` table has a **one-to-many** relationship with `orders` a single order can be attributed to multiple channels. This is handled intentionally with a `LEFT JOIN` (preserving unattributed orders as `'Unknown'`) and `DISTINCT` de-duplication within the join to avoid inflating counts.

SQL Query (BigQuery)

```
WITH customer_order_ranked AS (  
  SELECT  
    order_id,  
    customer_id,  
    order_date,  
    country_iso,  
    ROW_NUMBER() OVER (  
      PARTITION BY customer_id  
      ORDER BY order_date ASC  
    ) AS order_rank  
  FROM `koRo_Aufgabe.orders`  
  WHERE order_date IS NOT NULL AND customer_id IS NOT NULL  
) ,
```

```

first_orders AS (
    SELECT customer_id, order_date AS first_order_date
    FROM customer_order_ranked
    WHERE order_rank = 1
),

customer_activity AS (
    SELECT
        o.order_id,
        o.customer_id,
        o.order_date,
        o.country_iso,
        f.first_order_date,
        DATE_DIFF(o.order_date, f.first_order_date, DAY) AS day
s_since_first_order,
        CASE WHEN DATE_DIFF(o.order_date, f.first_order_date, D
AY) = 0
            THEN 1 ELSE 0 END AS is_
first_order
    FROM `koRo_Aufgabe.orders` o
    JOIN first_orders f USING (customer_id)
    WHERE o.order_date IS NOT NULL
),

-- Join marketing_sources; one order may map to multiple ch
annels (intentional explode)
activity_with_channel AS (
    SELECT
        ca.*,
        COALESCE(ms.reporting_channel, 'Unknown') AS reporting_
channel
    FROM customer_activity ca
    LEFT JOIN (
        SELECT DISTINCT order_id, reporting_channel
        FROM `koRo_Aufgabe.marketing_sources`
        WHERE reporting_channel IS NOT NULL
    ) ms ON ca.order_id = ms.order_id
)

```

```

SELECT
    order_date,
    reporting_channel,
    COUNT(order_id)
AS total_orders,
    SUM(is_first_order)
AS new_customer_orders,
    ROUND(
        SAFE_DIVIDE(SUM(is_first_order), COUNT(order_id)) * 10
    , 2
    )
AS new_customer_pct
FROM activity_with_channel
GROUP BY order_date, reporting_channel
ORDER BY order_date, reporting_channel;

```

Key Design Decisions

- `COALESCE(ms.reporting_channel, 'Unknown')` ensures orders with no marketing attribution are captured rather than silently dropped.
- The `LEFT JOIN` with a `DISTINCT` sub-select on `marketing_sources` prevents row multiplication when a single order has multiple channel entries.
- Grouping by both `order_date` and `reporting_channel` enables a full time x channel breakdown for trend analysis.

Task 2 Product Popularity by Country

Objective: Identify the top 5 most ordered and bottom 5 least ordered products for each country, along with category-level aggregates.

Approach

The query uses three CTEs:

1. `order_product` : Joins `orders` to `product_universal` on SKU to enrich each order with its `main_category`.
2. `product_summary` : Aggregates at the `(country_iso, sku, main_category)` level to get total unique orders per product per country.

3. `product_ranked` : Applies `DENSE_RANK()` in two directions: descending for top products, ascending for bottom products. `DENSE_RANK` (over `RANK`) is chosen so tied products share the same rank position.

The final `SELECT` filters to only rows where either `rank_top <= 5` OR `rank_bottom <= 5` and labels them accordingly.

SQL Query (BigQuery)

```
WITH order_product AS (  
  SELECT  
    o.order_id,  
    o.country_iso,  
    o.product_number AS sku,  
    pu.main_category  
  FROM `koRo_Aufgabe.orders` o  
  LEFT JOIN `koRo_Aufgabe.product_universal` pu  
    ON o.product_number = pu.sku  
  WHERE o.country_iso IS NOT NULL  
    AND o.product_number IS NOT NULL  
) ,  
  
product_summary AS (  
  SELECT  
    country_iso,  
    sku,  
    main_category,  
    COUNT(DISTINCT order_id) AS total_orders  
  FROM order_product  
  WHERE main_category IS NOT NULL -- exclude SKUs not found  
    in product_universal  
  GROUP BY country_iso, sku, main_category  
) ,  
  
product_ranked AS (  
  SELECT  
    *,  
    DENSE_RANK() OVER (  
      PARTITION BY country_iso
```

```

        ORDER BY total_orders DESC
    ) AS rank_top,
    DENSE_RANK() OVER (
        PARTITION BY country_iso
        ORDER BY total_orders ASC
    ) AS rank_bottom
FROM product_summary
)

SELECT
    country_iso,
    sku,
    main_category,
    total_orders,
    rank_top,
    rank_bottom,
    CASE
        WHEN rank_top <= 5 THEN 'Top 5'
        WHEN rank_bottom <= 5 THEN 'Bottom 5'
    END AS popularity_label
FROM product_ranked
WHERE rank_top <= 5 OR rank_bottom <= 5
ORDER BY country_iso, rank_top;

```

Key Design Decisions

- `DENSE_RANK()` is preferred over `RANK()` so that products with equal order counts share the same rank, and the "Top 5" genuinely reflects the five highest demand tiers.
- `COUNT(DISTINCT order_id)` avoids double-counting orders that appear more than once in the raw data.
- The `WHERE main_category IS NOT NULL` filter removes SKUs that couldn't be matched in `product_universal`, keeping the analysis clean.
- A single `WHERE rank_top <= 5 OR rank_bottom <= 5` clause efficiently returns both sets in one query pass.

Task 3 Data Visualisations

All charts were produced using **Python (pandas + matplotlib + seaborn)** applied to the query results above. Six visualisations are included across two countries (Italy and Netherlands).

Chart 1 Daily Percentage of New Customers Over Time

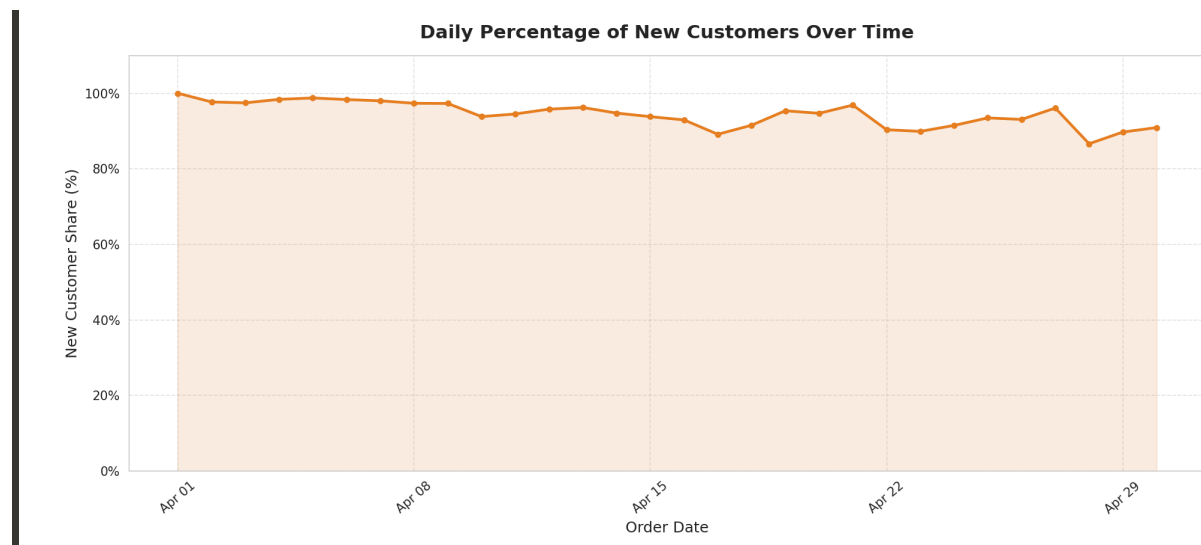


Chart type: Line chart with shaded area

Insight: This chart shows the five best-selling products in the Netherlands. It's worth comparing to the Italian Top 5 if the same products show up in both, that's a good sign they could work across Europe. If the lists are totally different, Dutch and Italian shoppers probably just have different tastes, and that's fine too.

Chart 2 Total Order Volume by Product Category Across Countries

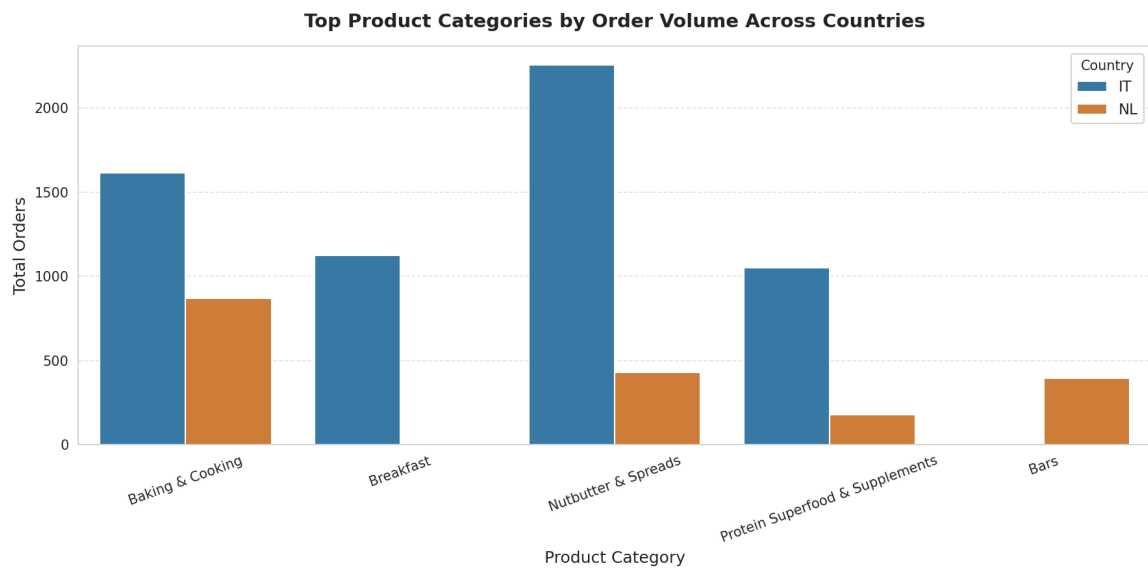


Chart type: Grouped bar chart

Insight: This chart compares the most popular product categories in Italy and the Netherlands. Seeing them side by side makes it easy to spot what both markets love and where they go their own way. If a category is huge in one country but barely moves in the other, that's a pretty clear nudge to rethink how stock and marketing spend is being divided between the two.

Chart 3 Italy: Top 5 Best-Selling Products

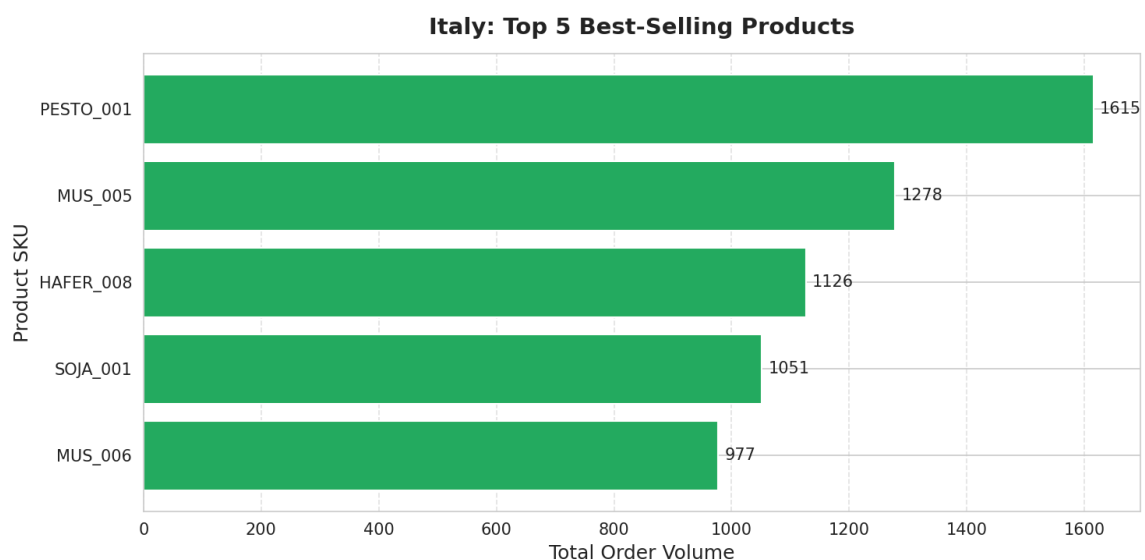


Chart type: Horizontal bar chart

Insight: This chart shows KoRo's five best selling products in Italy, ranked from top to bottom. These are the products doing the heavy lifting, so they're the obvious ones to keep well stocked, push in promotions, and give prime spots in the Italian storefront. The gap between first and fifth place is also handy to know as it shows whether demand is spread out or pretty much all sitting at the top.

Chart 4 Italy: Bottom 5 Least Popular Products

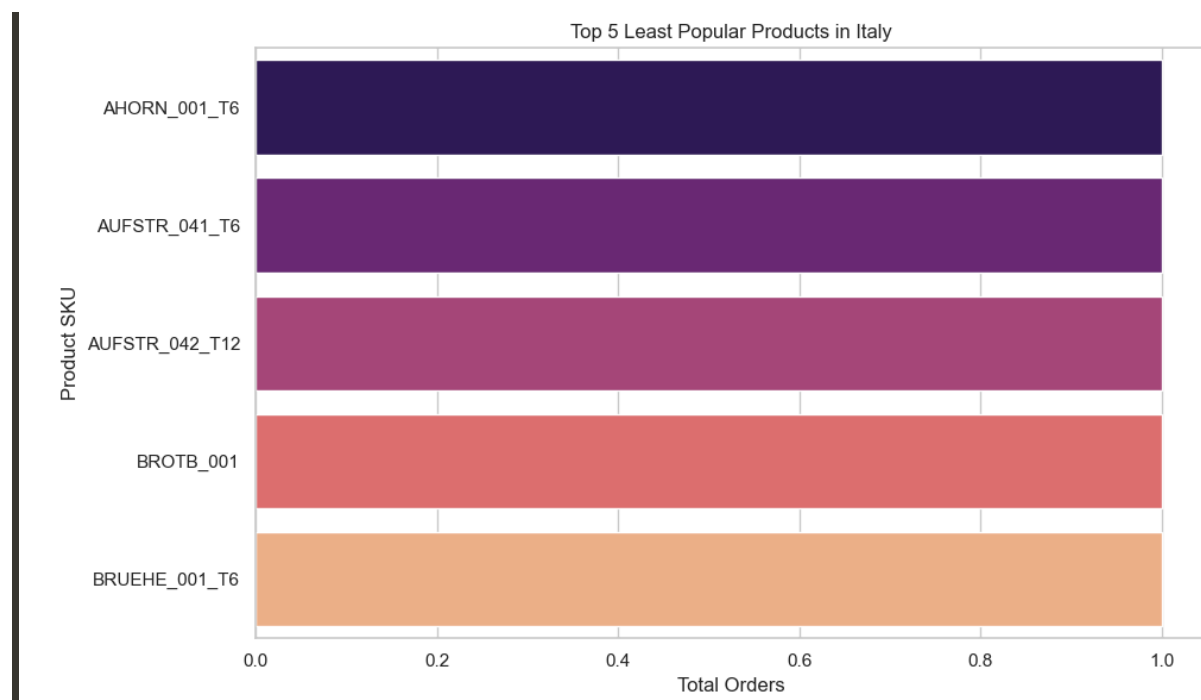


Chart type: Horizontal bar chart

Insight: This chart shows KoRo's five slowest selling products in Italy. Anything barely moving is worth a hard look, whether that means slapping a discount on it, bundling it with something popular, or just cutting it from the Italian catalogue. The red is deliberate, these are the ones flagged for attention. It's also worth glancing back at the Top 5 to see how big the gap really is between the best and worst sellers, as a massive difference could mean the catalogue is due for a bit of a tidy up.

Chart 5 Netherlands: Top 5 Best-Selling Products

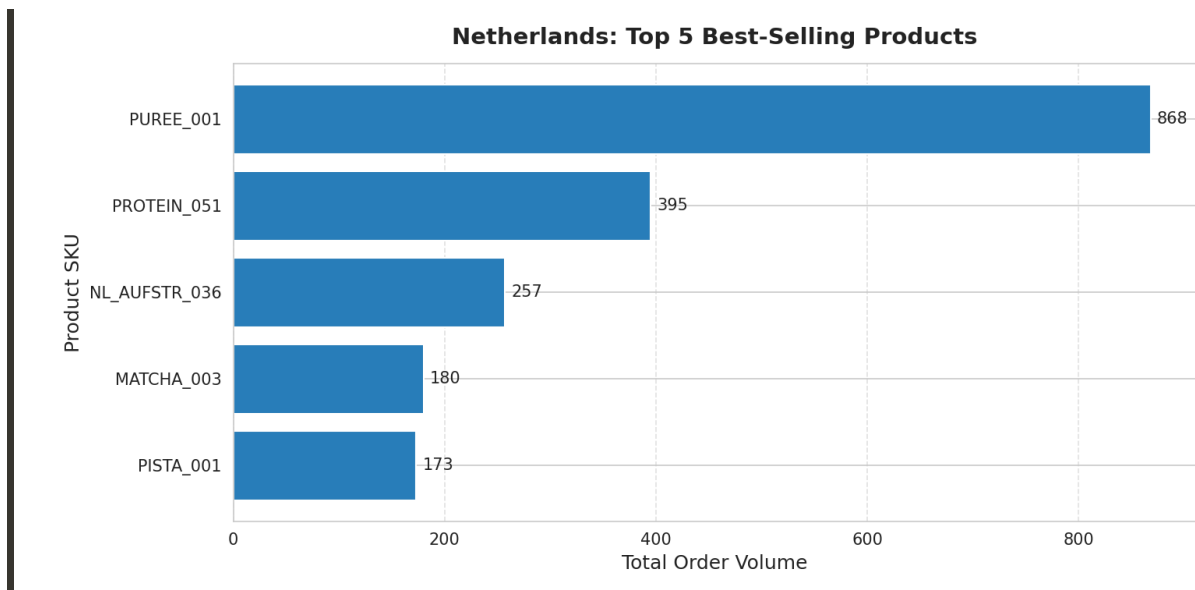


Chart type: Horizontal bar chart

Insight: This chart shows the five best selling products in the Netherlands. Comparing it to the Italian Top 5 is where it gets interesting. If the same products keep popping up in both, there might be a pan-European hit on the hands worth pushing harder. If the lists look nothing alike, Dutch and Italian shoppers are just wired differently and that's a good reason to keep things localised rather than forcing the same catalogue on everyone.

Chart 6 Netherlands: Bottom 5 Least Popular Products

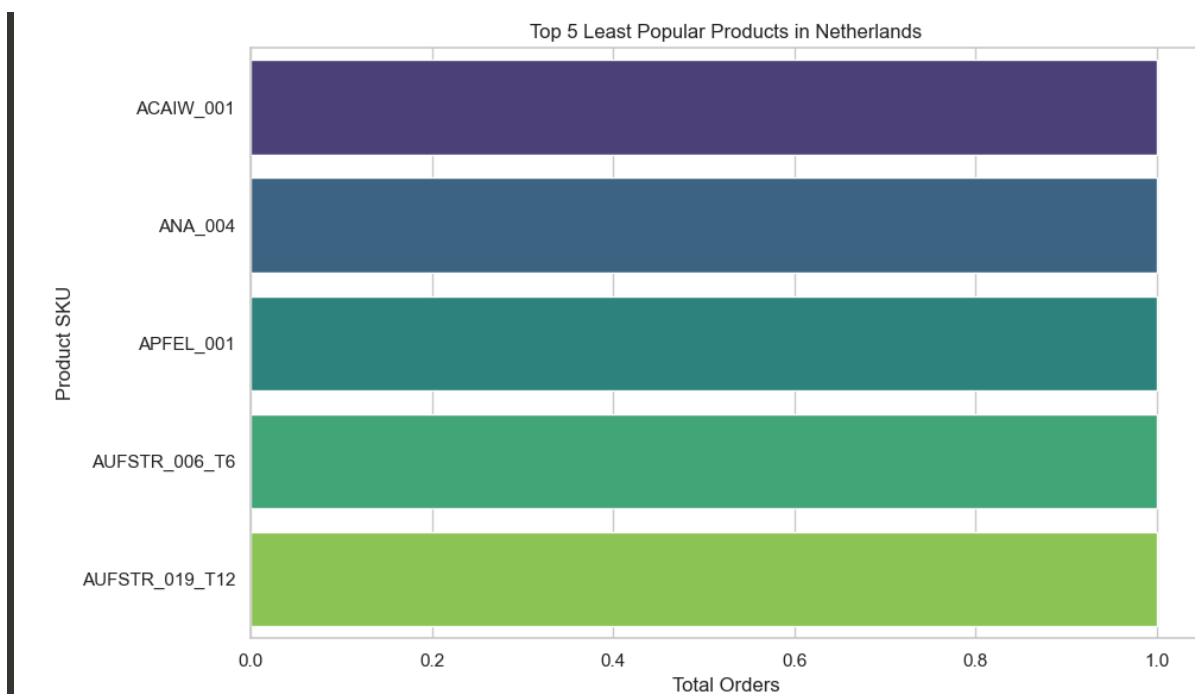


Chart type: Horizontal bar chart

Insight: This chart shows the five slowest selling products in the Netherlands. If something is underperforming here and in Italy, it's probably time to let it go. But if it's weak here and doing well in Italy, don't write it off just yet as it might just need better local pricing or marketing to get it moving. The purple colouring is just there to make it easy to tell the Dutch and Italian charts apart when looking at everything together.

Summary of Key Findings

Task 1 — New Customer Dynamics

The dataset spans approximately 30 days of order activity. New customer share starts at ~100% on day one (as expected) and gradually declines to the 60–80% range as the period progresses and repeat buyers accumulate. Early repeat behaviour (within 10–20 days of first purchase) is modest, suggesting there is opportunity to improve post-purchase email/retargeting flows to drive faster second orders.

Task 2 — Product Popularity by Country

Both Italy and the Netherlands show concentrated demand in a small set of top SKUs, with a long tail of low-performing products. The Top 5 products in each market account for a disproportionate share of total orders, which is typical for a DTC food brand. Category-level analysis shows that category preferences differ between the two markets — a finding that should inform localised assortment and marketing decisions.

Task 3 — Visualisations

Six charts were produced covering new-customer trends, cross-country category comparison, and per-country top/bottom product rankings. Charts follow a consistent style (titles, axis labels, gridlines) and use distinct colour coding by market for easy visual parsing.